Vertebra FEA Quickstart

This guide shows how to check out the Ogo FEA branch and run a solved vertebra compression model with FAIM/N88. It is intended for users who already have a density-calibrated CT image, a labelled vertebra segmentation, and access to the Numerics88 command-line tools.

What This Workflow Does

`ogoFEA spine` is the user-facing entry point for vertebral compression models. For each requested vertebra it:

1. Extracts the vertebral body and posterior process labels from a labelled spine mask. 2. Crops the calibrated image and masks around the target vertebra. 3. Aligns the vertebral body to the bundled Ogo reference with ICP. 4. Resamples image and masks to isotropic model spacing. 5. Builds superior and inferior PMMA supports on stable vertebral-body contact surfaces. 6. Converts density to finite-element material IDs. 7. Writes an `.n88model`, audits the boundary conditions, and records modeling metadata. 8. Solves the model with FAIM/N88 and writes a compact `_results.csv`. 9. Optionally runs full-model and masked-ROI Pistoia postprocessing.

Fresh Setup On A New Computer

Use this section when setting up a new workstation from scratch.

1. Get Ogo

Install Git and a conda-style Python distribution such as Miniforge or Miniconda, then check out the FEA branch:

```
git clone git@github.com:Bonelab/Ogo.git
cd Ogo
git checkout ogo-fea
git pull origin ogo-fea
```

2. Create The Ogo Environment

The most reliable setup is to create an environment that gets `vtkbone` from the Numerics88 conda channel. This avoids the slow full `environment.yml` solve and avoids the common plain-`requirements.txt` failure where `vtkbone` is missing.

```
conda create -n ogo -c numerics88 -c conda-forge \
  git pandas nibabel pyyaml vtkbone pbr "setuptools<69" wheel python=3
conda activate ogo
PBR_VERSION=0.0.1 pip install -e .
```

Then verify the command-line entry point:

```
which ogoFEA
ogoFEA --help
```

The repository also contains `environment.yml`, but it can take a long time to solve on a new machine. Use it only if you need to reproduce that exact pinned environment:

```
conda env create -n ogo-pinned -f environment.yml
```

If `pip install -e .` fails with a PBR versioning error, first check whether Git is available inside the active environment:

```
which git
```

If this prints nothing, install Git into the environment and rerun the editable install:

```
conda install -c conda-forge git
PBR_VERSION=0.0.1 pip install -e .
```

`PBR_VERSION=0.0.1` is only used for package metadata during installation. It does not change the checked-out Ogo code.

3. Make N88/FAIM Discoverable

Ogo builds the `.n88model` in Python, but solving and Pistoia require the Numerics88/FAIM command-line tools.

If the N88 commands are already on your `PATH`, this should print paths:

```
which n88modelinfo
which n88pistoia
which n88tabulate
which faim
```

If they are not on `PATH`, decide how Ogo should find them:

Setup	How to run
N88 tools live in one directory	Pass `--faim_bin_dir /path/to/n88/bin`.
N88 tools live in a separate conda environment	Pass `--faim_env ENV_NAME`.
Individual tools have unusual names or locations	Pass explicit overrides such as `--n88pistoia_command /path/to/n88pistoia`.

On a machine with a separate N88 environment, a typical check is:

```
conda run -n ogoloco-n88 n88modelinfo --help
conda run -n ogoloco-n88 n88pistoia --help
```

4. Verify With A Debug Model

Before the first solved run on a new computer, generate an unsolved debug model:

```
ogoFEA spine \
  sub-001_desc-vqct_ct.nii.gz \
  sub-001_desc-spine_labels.nii.gz \
  --vertebra L1:2:3 \
  --output_path derivatives/fea_test \
  --debug \
  --no-solve
```

This confirms that Ogo can read the images, find the labels, align the vertebra, build the PMMA supports, and write the `.n88model`. After inspecting the debug output, run the solved command below in a clean output directory.

Required Inputs

You need two images in the same image space:

Input	Requirement
Calibrated CT	Scalar density image in K2HPO4-equivalent units.
Labelled spine mask	Integer labels containing the vertebral body and posterior process.

The vertebra target is passed as:

```
LEVEL:BODY_LABEL:PROCESS_LABEL
```

For example, `L1:2:3` means:

Part	Meaning
`L1`	Output level name.
2	Label value for the L1 vertebral body.
3	Label value for the L1 posterior process.

If you want masked Pistoia for a region such as the vertebral body, provide a binary ROI mask in the original CT image space.

Standard L1 Command

This command generates, audits, solves, and postprocesses one L1 model:

```
ogoFEA spine \
  sub-001_desc-vqct_ct.nii.gz \
  sub-001_desc-spine_labels.nii.gz \
  --vertebra L1:2:3 \
  --output_path derivatives/fea \
  --threads 4 \
  --faim_bin_dir /path/to/n88/bin \
  --run_pistoia \
  --critical_volume 12 \
  --critical_strain 0.007
```

Use `--threads 4` for a typical single-model run. This controls FAIM solver threads for that one model; it does not launch multiple subjects in parallel.

Full And Masked Pistoia

To report both full-model Pistoia and an ROI-masked Pistoia result, add `--pistoia_mask`:

```
ogoFEA spine \
  sub-001_desc-vqct_ct.nii.gz \
  sub-001_desc-spine_labels.nii.gz \
  --vertebra L1:2:3 \
  --pistoia_mask sub-001_desc-L1Body_mask.nii.gz \
  --output_path derivatives/fea \
  --threads 4 \
  --faim_bin_dir /path/to/n88/bin \
  --run_pistoia \
  --critical_volume 12 \
  --critical_strain 0.007
```

With this command, Ogo reports:

Result	Description
Full-model Pistoia	Uses the solved vertebra model and excludes PMMA.
Masked Pistoia	Uses only elements selected by the supplied ROI mask after transformation into model space.

The same `--critical_volume` and `--critical_strain` values are used for both full-model and masked Pistoia in a single `ogoFEA` run.

N88 And FAIM Path Handling

Ogo does not search the entire filesystem for N88. The commands are resolved in this order:

1. Explicit command overrides such as `--faim_command` or `--n88pistoia_command`. 2. `--faim_bin_dir`, for example `/path/to/n88/bin`. 3. `--faim_install_root`, where Ogo checks common subdirectories. 4. The current shell `PATH`.

If the N88 tools are in a separate conda environment, use:

```
ogoFEA spine \
  sub-001_desc-vqct_ct.nii.gz \
  sub-001_desc-spine_labels.nii.gz \
  --vertebra L1:2:3 \
  --output_path derivatives/fea \
  --faim_env ogoloco-n88 \
  --threads 4
```

Expected Outputs

By default, outputs are written to the calibrated image directory. In most projects you should pass `--output_path derivatives/fea`.

Important outputs:

File	Meaning
`*_L1.n88model`	Generated finite-element model.
`*_L1_modeling.json`	Traceable metadata for inputs, preprocessing, materials, boundary conditions, and reporting.
`*_L1_results.csv`	Compact one-row results table for downstream analysis.
`*_L1_pistoia.txt`	Full-model Pistoia report, when Pistoia is run.
`*_L1_masked_pistoia.txt`	ROI-masked Pistoia report, when `--pistoia_mask` is supplied.
`*_L1_pistoia_mask.nii.gz`	ROI mask transformed into model space.

The `_results.csv` includes reaction force, stiffness, full-model Pistoia fields, and `masked_pistoia_*` fields when a mask is supplied.

Model Defaults

The default high-level spine preset is `benchmark-linear`.

Setting	Default
Model type	Linear elastic vertebral compression.
Resampling	1.0 mm isotropic.
Density-to-modulus law	`E = 2980 * rho^1.05 MPa` for benchmark-linear spine models.
Bone Poisson's ratio	0.3.
PMMA modulus	2500 MPa.
PMMA Poisson's ratio	0.3.
PMMA thickness	10 mm.
Default target endpoint	0.68% compression strain.
Full-model Pistoia critical strain	User-supplied; commonly `0.007`.

FAIM stores prescribed displacement in millimeters. Before solving, Ogo converts the 0.68% endpoint to millimeters from the generated vertebral-body height and updates the `.n88model`.

Quick Quality Checks

For a new dataset, first generate a debug model without solving:

```
ogoFEA spine \
  sub-001_desc-vqct_ct.nii.gz \
  sub-001_desc-spine_labels.nii.gz \
  --vertebra L1:2:3 \
  --output_path derivatives/fea \
  --debug \
  --no-solve
```

Check:

- The vertebral body is upright in the model frame.

- The posterior process is posterior to the body, not rotated into the axial

direction.

- Superior and inferior PMMA supports contact the body surfaces without being

dominated by osteophytes.

- The boundary-condition audit passes.

- The `_modeling.json` records the expected labels, material law, displacement

endpoint, and N88 settings.

Common Problems

Symptom	Likely cause	What to check
Command cannot find `faim` or `n88pistoia`	N88 tools are not on PATH.	Pass `--faim_bin_dir`, `--faim_env`, or explicit command paths.
Missing masked Pistoia fields	No `--pistoia_mask` was supplied, or masked Pistoia failed.	Check `_results.csv`, `*_masked_pistoia.txt`, and the log.
Process appears rotated relative to the body	ICP alignment or label definition problem.	Run `--debug --no-solve` and inspect the model before solving.
PMMA support contacts osteophytes instead of a stable surface	Endplate surface is irregular.	Inspect debug output and `_modeling.json`; consider excluding the case if QC fails.
Existing solution is reused	The `.n88model` already contains a solution.	Remove old outputs or write to a clean `--output_path` before rerunning.

Minimal Reproducible Example

For a collaborator, the most useful handoff is:

```
git clone git@github.com:Bonelab/Ogo.git
cd Ogo
git checkout ogo-fea
python -m pip install -e .

ogoFEA spine \
  image_K2HPO4.nii.gz \
  spine_labels.nii.gz \
  --vertebra L1:2:3 \
  --pistoia_mask L1_body_mask.nii.gz \
  --output_path derivatives/fea \
  --threads 4 \
  --faim_bin_dir /path/to/n88/bin \
  --run_pistoia \
  --critical_volume 12 \
  --critical_strain 0.007
```

The main file to send onward for tabulated analysis is `derivatives/fea/*_L1_results.csv`; the main file for visual/model inspection is `derivatives/fea/*_L1.n88model`.